ISA 632 Group Project

Execution Instructions

Cloud-Based GenAI Chatbot with Databricks

Purpose. This guide explains how your team should execute the group project by adapting the four labs to a new business or real-life use case. The goal is not to invent a completely new technical stack. The goal is to reuse the lab workflow thoughtfully, customize it for a meaningful problem, and demonstrate a working GenAI chatbot prototype.

Project Overview

Identify a useful chatbot scenario, prove why a generic LLM is not enough, connect the model to relevant context data through RAG, evaluate its responses, deploy it, and present it professionally as a consultant team.

1. Expected Project Scope

Your project should be realistic and manageable. A strong project does not require massive data collection or advanced model training. It does require a clear business problem, relevant context data, a functional RAG-based chatbot, meaningful evaluation, and a polished demo.

- **Required core components:** business problem, LLM boundary test, dataset/context source, embeddings and retrieval, LangChain/RAG model, system prompt, evaluation, deployment, and demo.
- **Recommended scope:** start with a small sample, such as around 1,000 rows or a limited collection of documents, before scaling up.
- **Not recommended:** large-scale scraping, overly complex data pipelines, or spending most of your time on data collection instead of building and testing the chatbot.
- **Mindset:** act like a small consulting team presenting a useful prototype to a client.

2. Relationship to the Four Labs

The four labs are your technical template. Your team should follow the same logic and reuse code where appropriate, but replace the original example data and prompts with your own business case.

Lab	What You Practiced	How It Supports the Project	What to Customize
Lab 1: Prompt Engineering	Test model behavior in AI Playground; identify hallucination, missing knowledge, and prompt limitations.	Use it to prove why a generic LLM is insufficient for your selected problem.	Try realistic user questions from your business case and document where the current model is weak.
Lab 2: Build and Register a RAG Model	Create a vector search index, retrieve documents, build a RAG pipeline, and register the model.	This is the main technical foundation for your customized chatbot.	Replace sample data with your dataset; update table names, index names, retriever logic, and prompt.
Lab 3: Evaluation with MLflow	Load the model, define built-in/custom metrics, create test questions, and evaluate results.	Use it to show whether the chatbot works and where it still fails.	Create representative evaluation questions and metrics for your scenario.

Lab 4: Real-Time Deployment	Deploy the registered model with Model Serving and test it through an app/review interface.	Use it to prepare the final demo and collect feedback.	Deploy your model version and test common user questions before presentation.

3. Step-by-Step Execution Plan

Step 1. Identify a business or real-life problem

Choose a scenario where a chatbot can help users answer questions using specific context information. Good examples include customer support, product support, advising, policy explanation, event information, restaurant or hotel review exploration, technical documentation help, or internal knowledge assistance.

What to produce: Your problem should be narrow enough to build in the remaining time. Avoid vague ideas such as "a chatbot for everything." Define the target users, their pain points, and 5-10 realistic questions they would ask.

Step 2. Test the boundary of existing LLMs

Before building your customized model, test whether a generic LLM can already solve the problem. Usually choosing an open-source model (such as GPT OSS, Llama, Qwen and Gemma on Databricks AI Gateways) is easier to identify the model gaps. Use questions that require current, local, private, domain-specific, or document-specific information.

What to produce: Document several cases where the model gives incomplete, outdated, generic, or fabricated answers. This becomes your justification for RAG and customization.

Step 3. Identify a dataset or document collection

Find a dataset that can serve as context for your chatbot. Public datasets on Internet and free datasets from Databricks Marketplace are preferred. You may also reuse a dataset from your ISA 514 project. The instructor may provide sample datasets and problem ideas for groups that struggle.

What to produce: The dataset does not need to be fancy. It only needs to contain useful text or semi-structured information that the chatbot can retrieve. Examples include product descriptions, FAQ pages, reviews, policy documents, public reports, documentation, PDFs, or CSV files with textual fields.

Step 4. Load the data into Databricks and build embeddings

Upload or connect the data in Databricks, clean it as needed, generate embeddings, and build a Vector Search index.

What to produce: Start small. Verify that your index works before scaling. Manually test several search queries and inspect the retrieved documents. Do not move forward until retrieval quality is reasonable.

Step 5. Build the LangChain agent/RAG model and system prompt

Adapt the Lab 2 code to your use case. The model should retrieve relevant context and use it to answer user questions. Design a system prompt that clearly defines the chatbot role, scope, tone, and constraints.

What to produce: Your prompt should tell the model what to do when context is missing, how to avoid hallucination, and how to respond in a useful business style. The model should not pretend to know information that is not available.

Step 6. Conduct evaluation

Adapt the Lab 3 workflow. Create representative test questions and judge response quality using built-in metrics, custom metrics and manual review.

What to produce: The evaluation does not need to be comprehensive. It should show deep thinking. Include normal questions, edge cases, questions with missing context, and questions that test whether the model uses the retrieved context correctly.

Step 7. Deploy and test the app

Adapt the Lab 4 workflow. Deploy the model through Databricks Model Serving or a similar approved interface. Test the deployed app and prepare the demo.

What to produce: If real-time demo is risky, record a backup video. Your presentation should still show that the model works and explain what is happening behind the scenes.

4. Minimum Required Evidence

Your final submission and presentation should provide evidence that each major component works. Use screenshots, notebook outputs, examples, tables, or demo clips where appropriate.

Component	Minimum Evidence	Suggested Format
Problem framing	Clear description of the user, business problem, chatbot purpose, and why GenAI is useful.	Slide, screenshot, notebook output, or short table
LLM boundary test	Examples showing where a generic LLM gives weak, generic, outdated, or hallucinated answers.	Slide, screenshot, notebook output, or short table
Dataset/context source	Description of the dataset, source, size, fields, and why it is relevant.	Slide, screenshot, notebook output, or short table
Retrieval quality check	Several example search queries and the retrieved context results. Include at least one case where retrieval works well and one case where it is imperfect.	Slide, screenshot, notebook output, or short table
RAG model	Working code that connects user questions, retrieval, prompt, and LLM response.	Slide, screenshot, notebook output, or short table
System prompt	A clear prompt that defines role, task, answer style, boundaries, and fallback behavior.	Slide, screenshot, notebook output, or short table
Evaluation	Representative test questions, evaluation criteria, and results. Include limitations.	Slide, screenshot, notebook output, or short table
Deployment/demo	Runnable chatbot or app, plus a backup demo plan if live deployment is unstable.	Slide, screenshot, notebook output, or short table

5. Dataset Guidance

- **Good datasets:** contain useful text, are easy to explain, and connect directly to user questions.
- **Acceptable formats:** CSV is preferred. Other formats, such as JSON, text files, PDF files, website documentation, product catalogs, public reviews, reports, manuals, policy documents, or FAQ pages, may need extra work.
- **Recommended size:** begin with a small sample. A few hundred to a few thousand records may be enough for a strong prototype.
- **Important fields:** identify the text field to embed, metadata fields for filtering or citation, and any IDs or titles that help users understand retrieved results.

- **Avoid:** datasets with no meaningful text, massive unorganized files, data requiring complicated scraping, or sensitive/private data without permission.

6. Retrieval Quality Checklist

Before focusing on model answers, verify that retrieval works. A poor retriever usually leads to poor chatbot responses even when the LLM is strong.

- Run at least 5-10 realistic search queries against your vector index.
- Inspect the top retrieved records manually.
- Check whether the retrieved context directly answers the query.
- Try different wording for the same question to see if retrieval is stable.
- Check for duplicated or nearly duplicated chunks.
- Adjust chunking, text fields, metadata, or query design if retrieval is weak (optional advanced feature).
- Record at least a few examples for your final presentation or technical document.

7. System Prompt Design Requirements

Your system prompt should be customized for your use case. Do not simply reuse a generic prompt without editing it.

- Define the chatbot role, such as customer service assistant, advising assistant, product support assistant, or documentation assistant.
- Define the target users and the type of help the chatbot should provide.
- Tell the model to use retrieved context as the primary source of information.
- Tell the model what to do when the retrieved context is insufficient.
- Include business-appropriate tone and formatting expectations.
- Include safety or policy boundaries when relevant. (optional advanced feature)
- Encourage concise, useful answers rather than long generic responses.

8. Evaluation Guidance

Evaluation should demonstrate that your team has thought carefully about quality. You do not need a large benchmark, but your test set should be representative.

- **Normal questions:** questions the chatbot should answer well.
- **Context-required questions:** questions that generic LLMs may not answer correctly without your dataset.
- **Missing-information questions:** questions where the correct behavior is to say the answer is not available.
- **Boundary questions:** questions outside the chatbot scope.
- **Suggested metrics:** correctness, groundedness, relevance, helpfulness, answer completeness, refusal quality, and retrieval quality.

9. Ways to Make Your Project Stand Out

The following enhancements are optional. They can help you differentiate your project.

Enhancement	What It Means
Advanced data types	Use non-CSV data such as PDFs, documentation pages, or other text sources.
Advanced chunking	Experiment with chunk size, overlap, document sections, or semantic chunking.
Re-ranking	Add a second-stage ranking step to improve retrieved context quality.
Deduplication	Remove repeated or near-duplicate chunks to reduce noisy retrieval.
Agentic AI	Allow the chatbot to use tools, route questions, or perform simple multi-step reasoning.
Multi-agent design	Use separate agents for retrieval, evaluation, summarization, or recommendation.
Fine-tuning	Only attempt this if your team has a strong technical background and enough time.
User interface polish	Create a cleaner app interface, Streamlit app, or demo workflow.
Feedback loop	Collect human feedback and explain how it could improve future versions.

10. Final Deliverables

- **Presentation slides:** professional, consultant-style, focused on problem, solution, evidence, demo, and impact.
- **Working chatbot/app:** a runnable prototype or deployed model that can answer realistic user questions.
- **Technical document:** brief explanation of data collection, preprocessing, vector store, prompt, model pipeline, evaluation, deployment, and limitations.
- **Companion files:** notebooks, scripts, data samples, prompt files, evaluation files, and any setup instructions needed to replicate the work.
- **Demo backup:** recorded video or screenshots in case the live app is unstable during presentation.

11. Suggested Presentation Structure

- Problem and client context: Who is the chatbot for, and what problem does it solve?
- Why generic LLMs are not enough: Show boundary test examples.
- Data and context engineering: Explain your dataset and how you prepared it.
- Architecture: Show the RAG pipeline at a high level.
- Demo: Ask realistic questions and show the chatbot response.
- Evaluation: Explain test questions, metrics, and key findings.
- Business value: Explain impact, usefulness, and limitations.
- Future work: Explain what you would improve with more time.

12. Common Pitfalls to Avoid

- Choosing a problem so broad that the chatbot cannot be evaluated clearly.
- Using a dataset only because it is available, not because it supports the use case.
- Skipping manual retrieval checks and only looking at final generated answers.
- Focusing on optional advanced features while the core RAG pipeline is weak.

13. Recommended Timeline

Phase	Team Tasks	Checkpoint Date
Planning and Data Setup	Select problem and prepare dataset.	April 26th
Model building and Evaluation	Build vector index, create agent model, run evaluation, and review performance.	May 3rd
Deployment and Refinement	Deploy model and test app. Further improve your agent with advanced features (optional)	May 10th
Presentation (10 minutes)	Create slides, rehearse, assign speaking roles.	May 11th for Sec A May 13th for Sec B
Final Submission	Deliver finalized Agent model and submit all required companion files.	May 15th